

Loja_Etim

Trabalhando com Imagens

```
PROJETO UPLOAD DE IMAGENS COM PHP - PARTE 1
localhost/phpmyadmin/server_sql.php

22
23 /*
24
25     PRODUTO          IMAGENS
26
27     1 calça jeans    ----->  N
28                               1
29                               I
30
31
32
```

Teremos as tabelas produto e imagem, um produto pode ter varias imagens mas essas imagens so podem pertencer a um produto

Para o projeto, quem não fez ainda, crie uma pasta com seu nome dentro de
c:\xampp\htdocs

Dentro dessa pasta crie uma pasta chamada Loja_Etim

Dentro dessa pasta crie as pastas

css

imagens

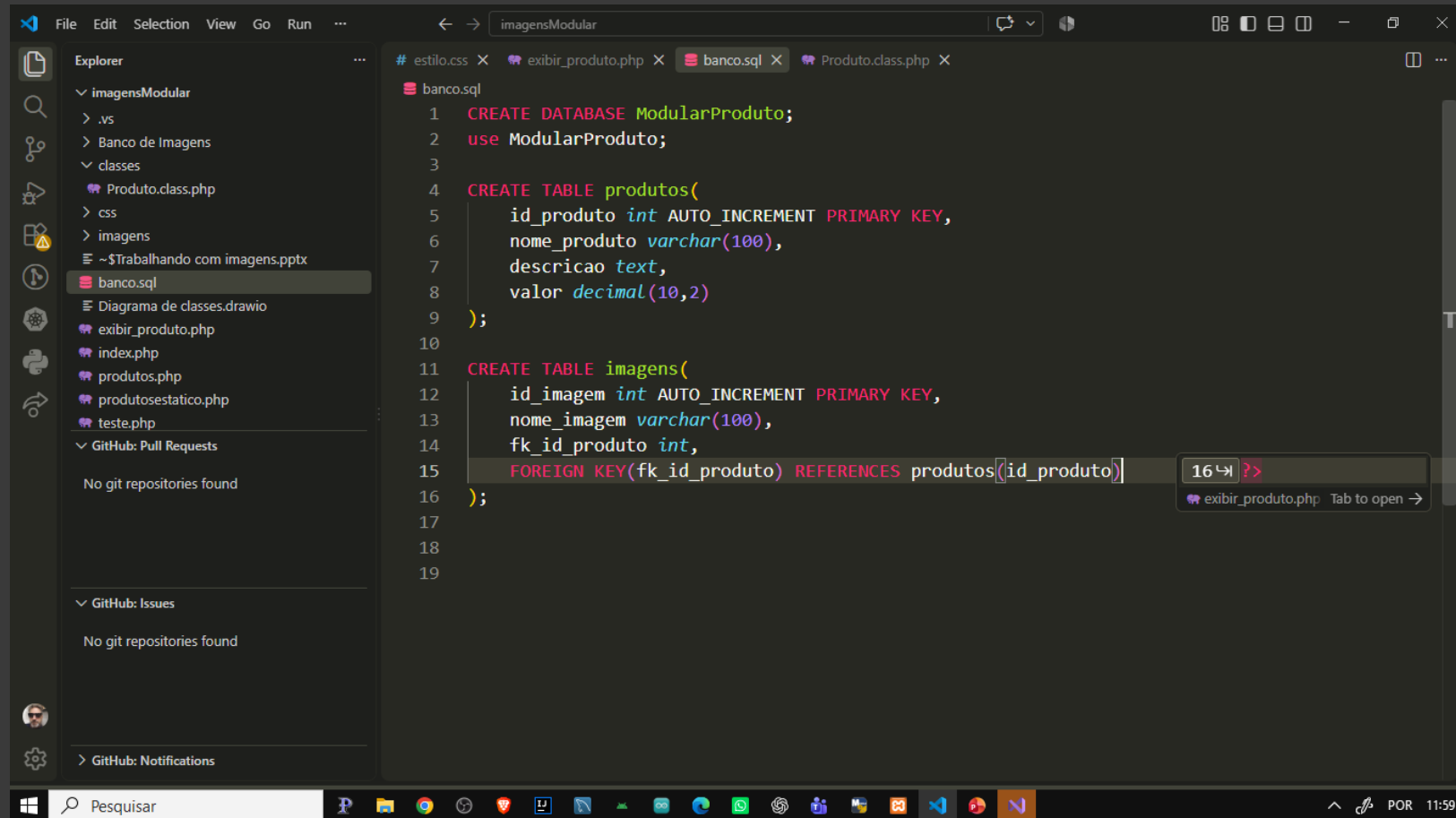
classes

sql

Crie o banco dentro da pasta css

Crie a classe Produto dentro da pasta classes

Crie o estilo dentro da pasta css



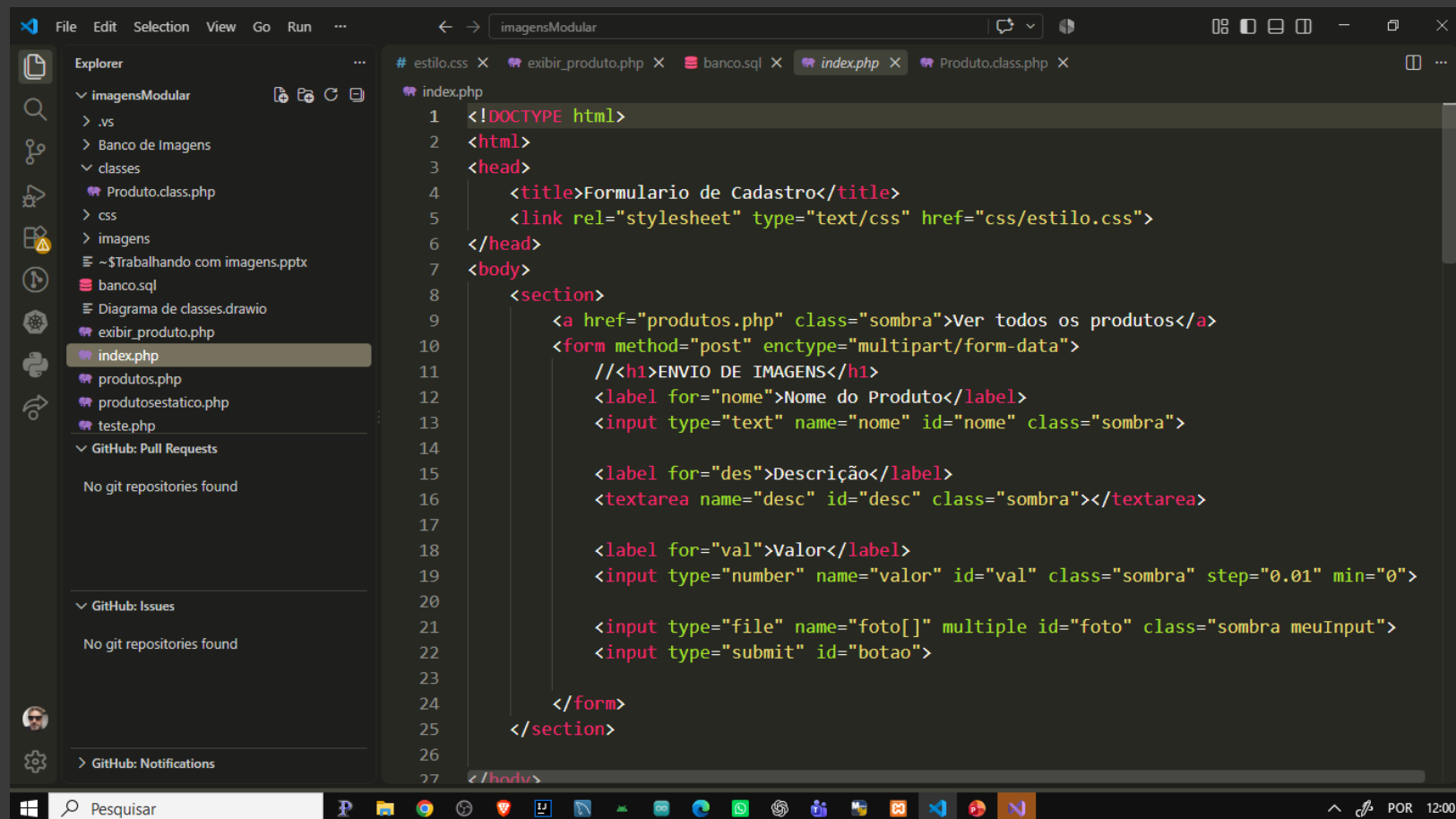
The image shows a Visual Studio Code editor window with the following content:

- Explorer Panel:** Shows the project structure for 'imagensModular'. It includes folders for '.vs', 'Banco de Imagens', 'classes', 'css', and 'imagens'. The 'banco.sql' file is selected in the 'imagens' folder. Below the file explorer, there are sections for 'GitHub: Pull Requests' and 'GitHub: Issues', both showing 'No git repositories found'.
- Editor Panel:** Displays the 'banco.sql' file with the following SQL code:

```
1 CREATE DATABASE ModularProduto;
2 use ModularProduto;
3
4 CREATE TABLE produtos(
5     id_produto int AUTO_INCREMENT PRIMARY KEY,
6     nome_produto varchar(100),
7     descricao text,
8     valor decimal(10,2)
9 );
10
11 CREATE TABLE imagens(
12     id_imagem int AUTO_INCREMENT PRIMARY KEY,
13     nome_imagem varchar(100),
14     fk_id_produto int,
15     FOREIGN KEY(fk_id_produto) REFERENCES produtos(id_produto)
16 );
17
18
19
```

- Run and Debug Panel:** A small tooltip is visible over line 15, showing the text '16 ↵ ?>' and a link to 'exibir_produto.php Tab to open →'.
- Taskbar:** The Windows taskbar is visible at the bottom, showing various application icons and the system clock indicating 'POR 11:59'.

Normalmente eu crio o banco por aqui para evitar que tenha de refazer caso alguém apague. Essa eh a estrutura do banco e tabelas.



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Formulario de Cadastro</title>
5   <link rel="stylesheet" type="text/css" href="css/estilo.css">
6 </head>
7 <body>
8   <section>
9     <a href="produtos.php" class="sombra">Ver todos os produtos</a>
10    <form method="post" enctype="multipart/form-data">
11      //<h1>ENVIO DE IMAGENS</h1>
12      <label for="nome">Nome do Produto</label>
13      <input type="text" name="nome" id="nome" class="sombra">
14
15      <label for="des">Descrição</label>
16      <textarea name="desc" id="desc" class="sombra"></textarea>
17
18      <label for="val">Valor</label>
19      <input type="number" name="valor" id="val" class="sombra" step="0.01" min="0">
20
21      <input type="file" name="foto[]" multiple id="foto" class="sombra meuInput">
22      <input type="submit" id="botao">
23
24    </form>
25  </section>
26
27 </body>
```

Esse é o index.php com o formulário. Cada aluno pode fazer o seu, mantendo os inputs.

Atencao: no form, colocar o atributo enctype. Sem ele não funciona o input file. No campo input file, colocar os atributos multiple e o name tem que ser um array para receber vários arquivos.

//

ENVIO DE IMAGENS

Ver todos os produtos

Nome do Produto

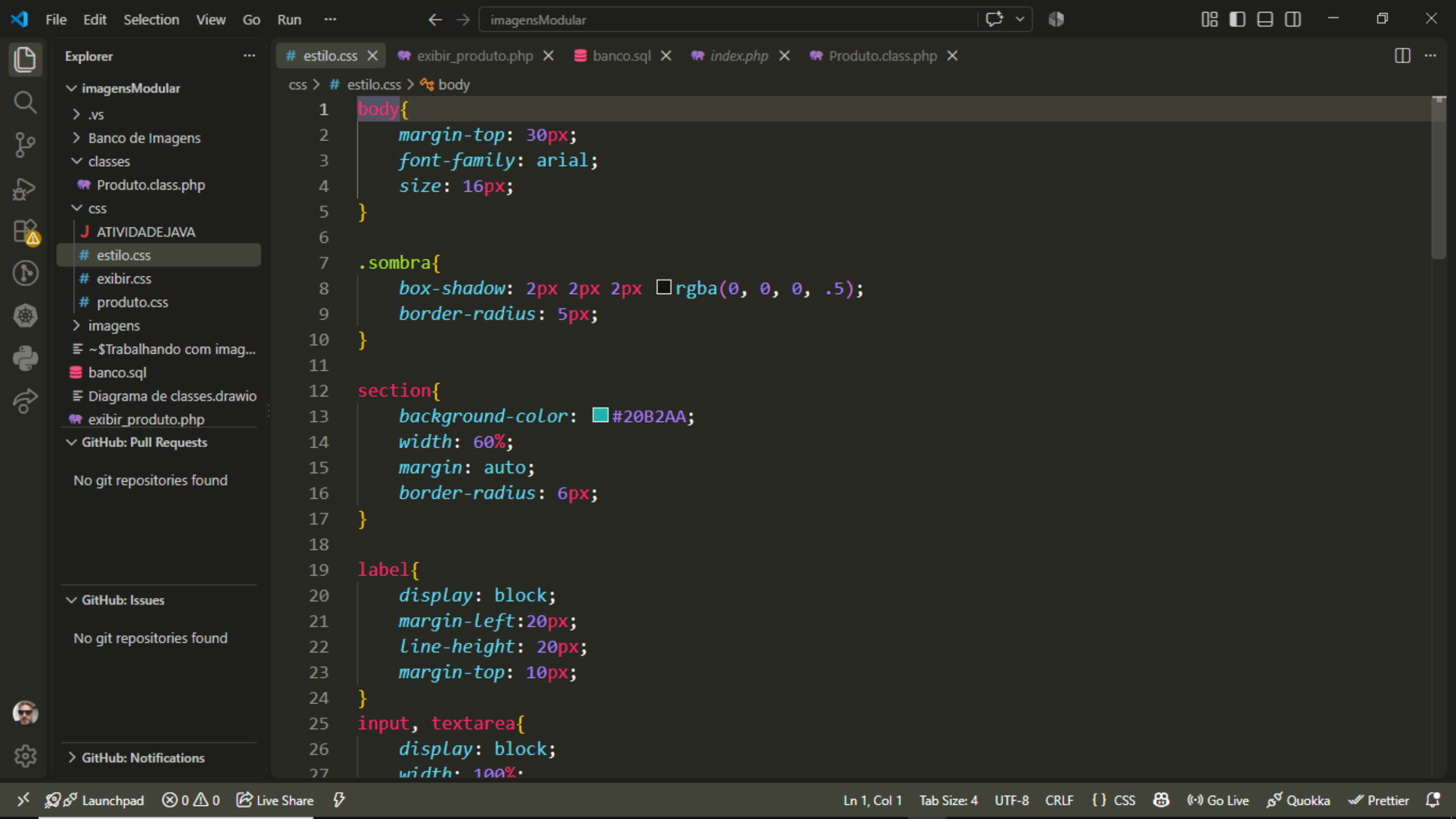
Descrição

Valor

Escolher arquivos Nenhum arquivo escolhido

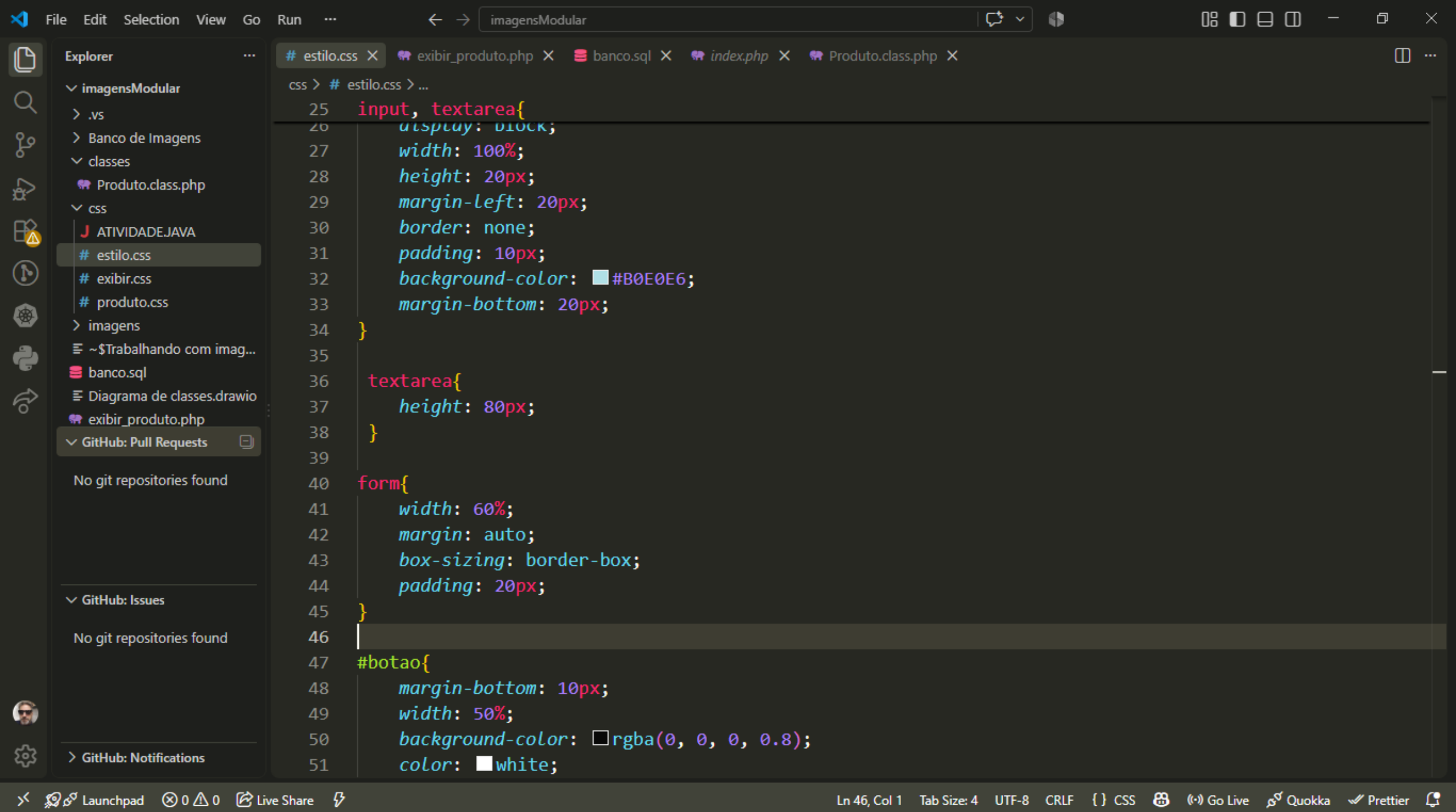
Enviar

Esse formulário, aplicando o estilo ficara dessa forma. Os alunos podem usar o estilo ou criar seu próprio. Nos próximos slides coloquei o css dessa pagina.

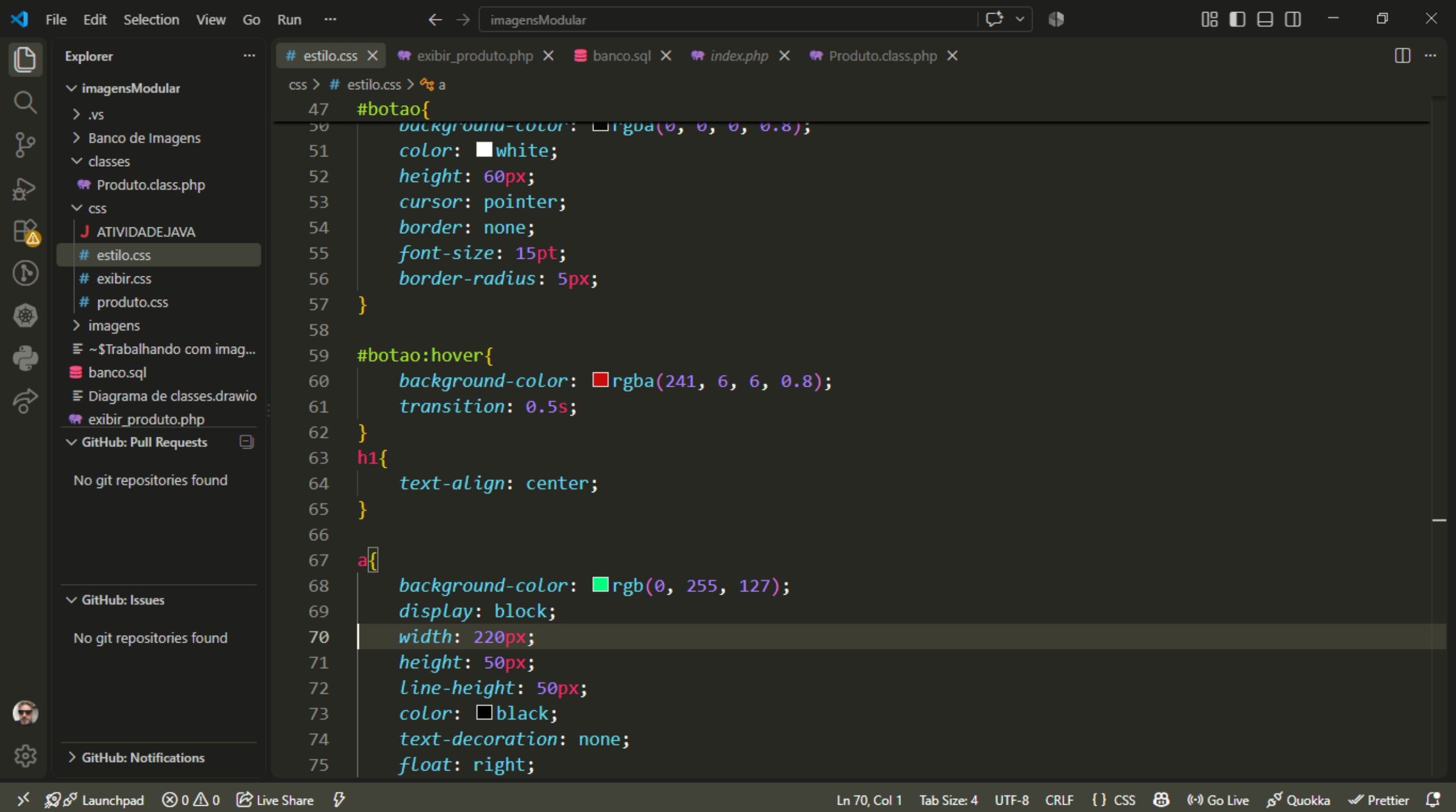


```
File Edit Selection View Go Run ...
# estilo.css x exibir_produto.php x banco.sql x index.php x Produto.class.php x

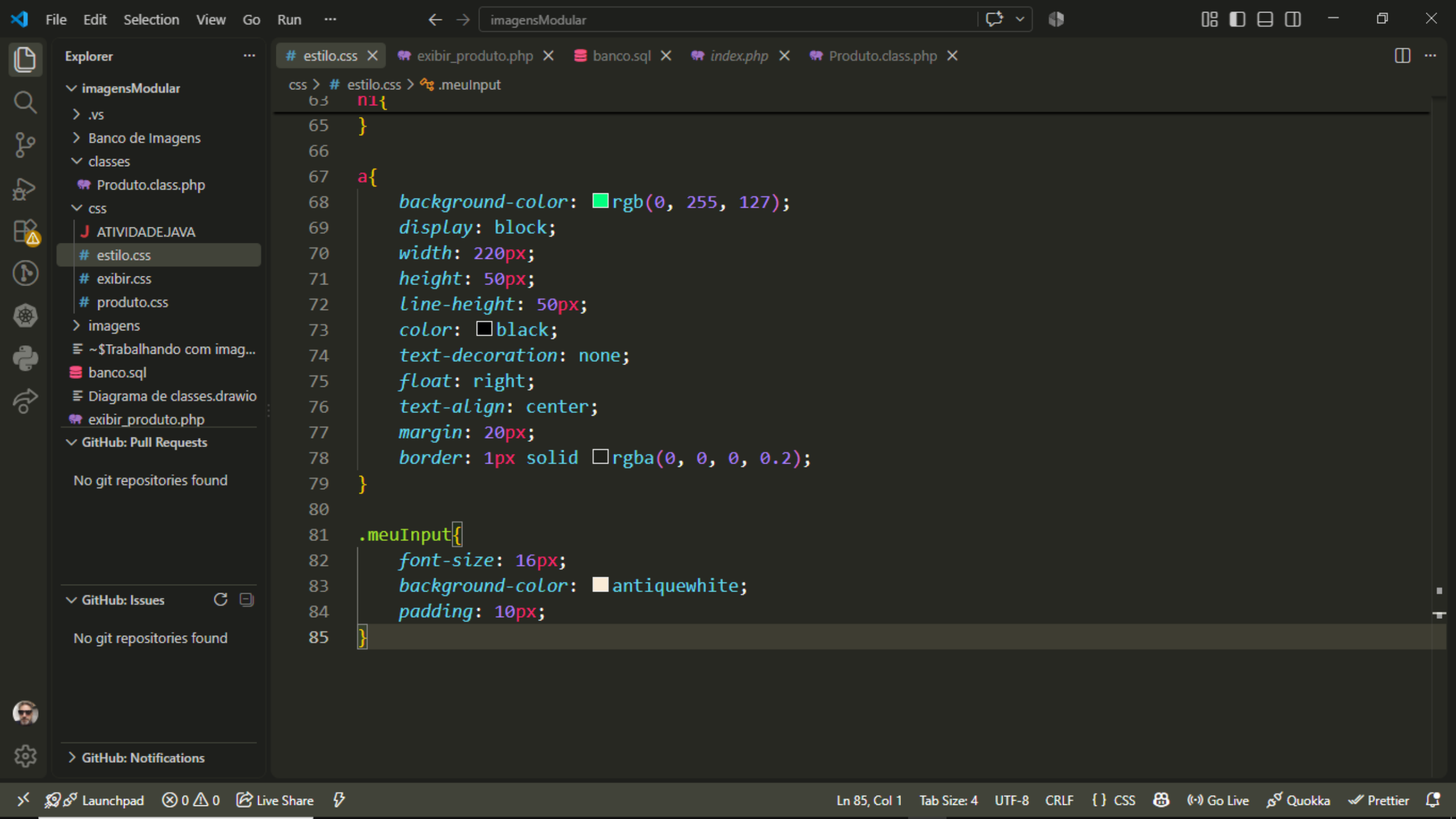
css > # estilo.css > body
1 body{
2     margin-top: 30px;
3     font-family: arial;
4     size: 16px;
5 }
6
7 .sombra{
8     box-shadow: 2px 2px 2px rgba(0, 0, 0, .5);
9     border-radius: 5px;
10 }
11
12 section{
13     background-color: #20B2AA;
14     width: 60%;
15     margin: auto;
16     border-radius: 6px;
17 }
18
19 label{
20     display: block;
21     margin-left: 20px;
22     line-height: 20px;
23     margin-top: 10px;
24 }
25 input, textarea{
26     display: block;
27     width: 100%;
```



```
css > # estilo.css > ...
25  input, textarea{
26      display: block;
27      width: 100%;
28      height: 20px;
29      margin-left: 20px;
30      border: none;
31      padding: 10px;
32      background-color: #B0E0E6;
33      margin-bottom: 20px;
34  }
35
36  textarea{
37      height: 80px;
38  }
39
40  form{
41      width: 60%;
42      margin: auto;
43      box-sizing: border-box;
44      padding: 20px;
45  }
46
47  #botao{
48      margin-bottom: 10px;
49      width: 50%;
50      background-color: rgba(0, 0, 0, 0.8);
51      color: white;
```

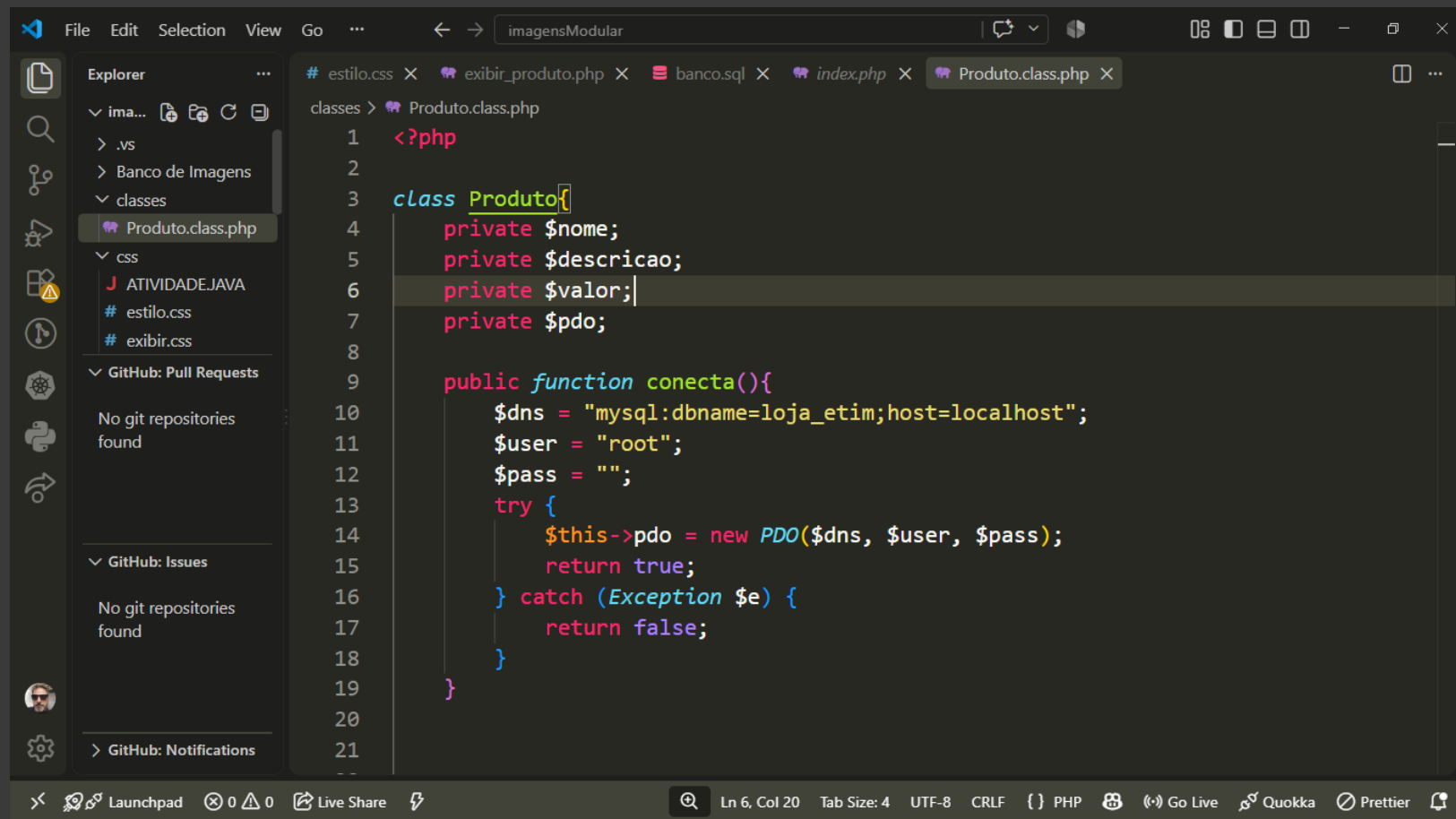



```
# estilo.css > a
47 #botao{
50     background-color: rgba(0, 0, 0, 0.8);
51     color: white;
52     height: 60px;
53     cursor: pointer;
54     border: none;
55     font-size: 15pt;
56     border-radius: 5px;
57 }
58
59 #botao:hover{
60     background-color: rgba(241, 6, 6, 0.8);
61     transition: 0.5s;
62 }
63 h1{
64     text-align: center;
65 }
66
67 a{
68     background-color: rgb(0, 255, 127);
69     display: block;
70     width: 220px;
71     height: 50px;
72     line-height: 50px;
73     color: black;
74     text-decoration: none;
75     float: right;
```



```
File Edit Selection View Go Run ...
# estilo.css x exibir_produto.php x banco.sql x index.php x Produto.class.php x

css > # estilo.css > .meuInput
63 n1{
65 }
66
67 a{
68     background-color: rgb(0, 255, 127);
69     display: block;
70     width: 220px;
71     height: 50px;
72     line-height: 50px;
73     color: black;
74     text-decoration: none;
75     float: right;
76     text-align: center;
77     margin: 20px;
78     border: 1px solid rgba(0, 0, 0, 0.2);
79 }
80
81 .meuInput{
82     font-size: 16px;
83     background-color: antiquewhite;
84     padding: 10px;
85 }
```

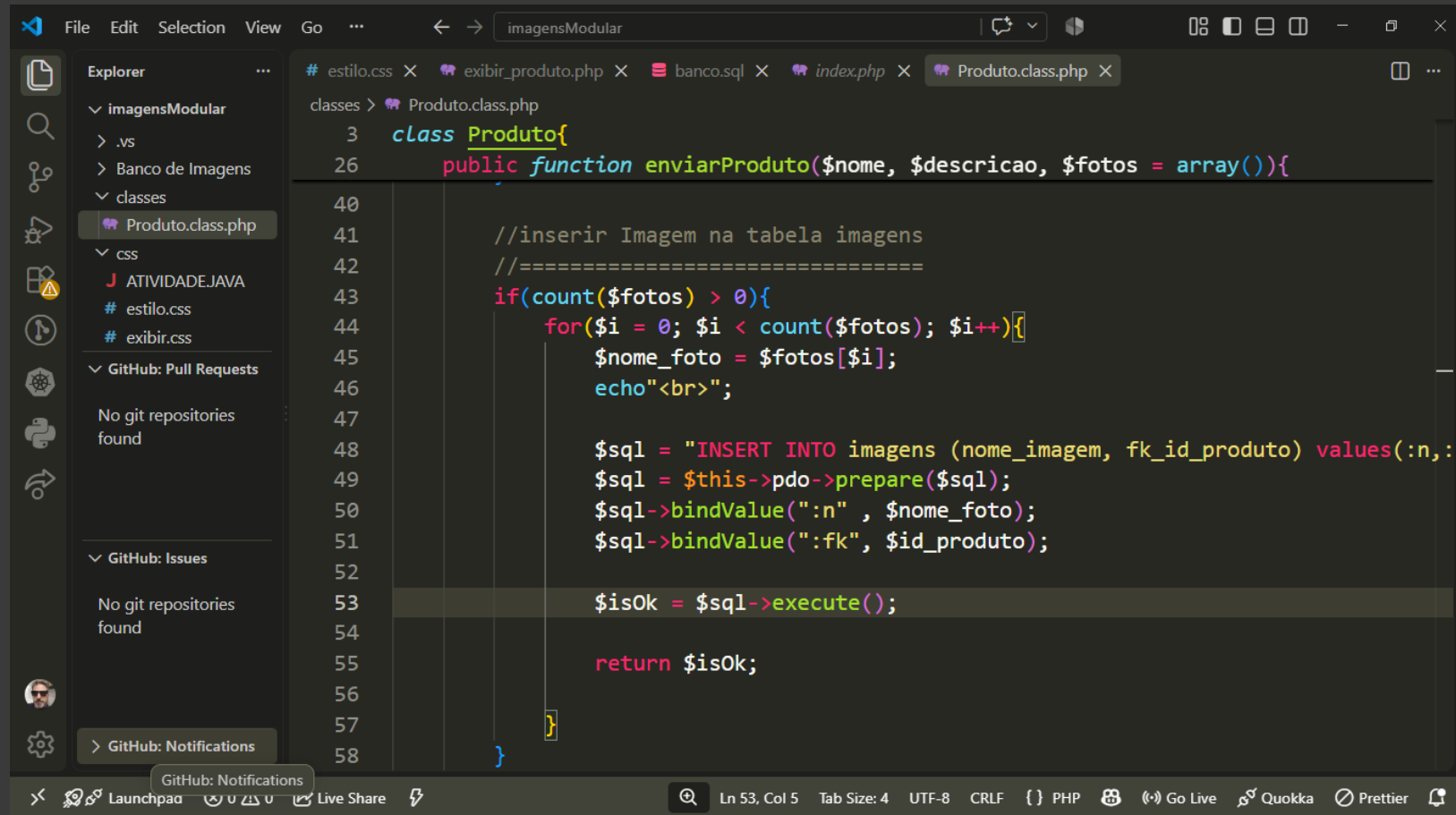


```
1  <?php
2
3  class Produto{
4      private $nome;
5      private $descricao;
6      private $valor;
7      private $pdo;
8
9      public function conecta(){
10         $dns = "mysql:dbname=loja_etim;host=localhost";
11         $user = "root";
12         $pass = "";
13         try {
14             $this->pdo = new PDO($dns, $user, $pass);
15             return true;
16         } catch (Exception $e) {
17             return false;
18         }
19     }
20
21
```

O método para
conectar com o banco
usando a classe PDO

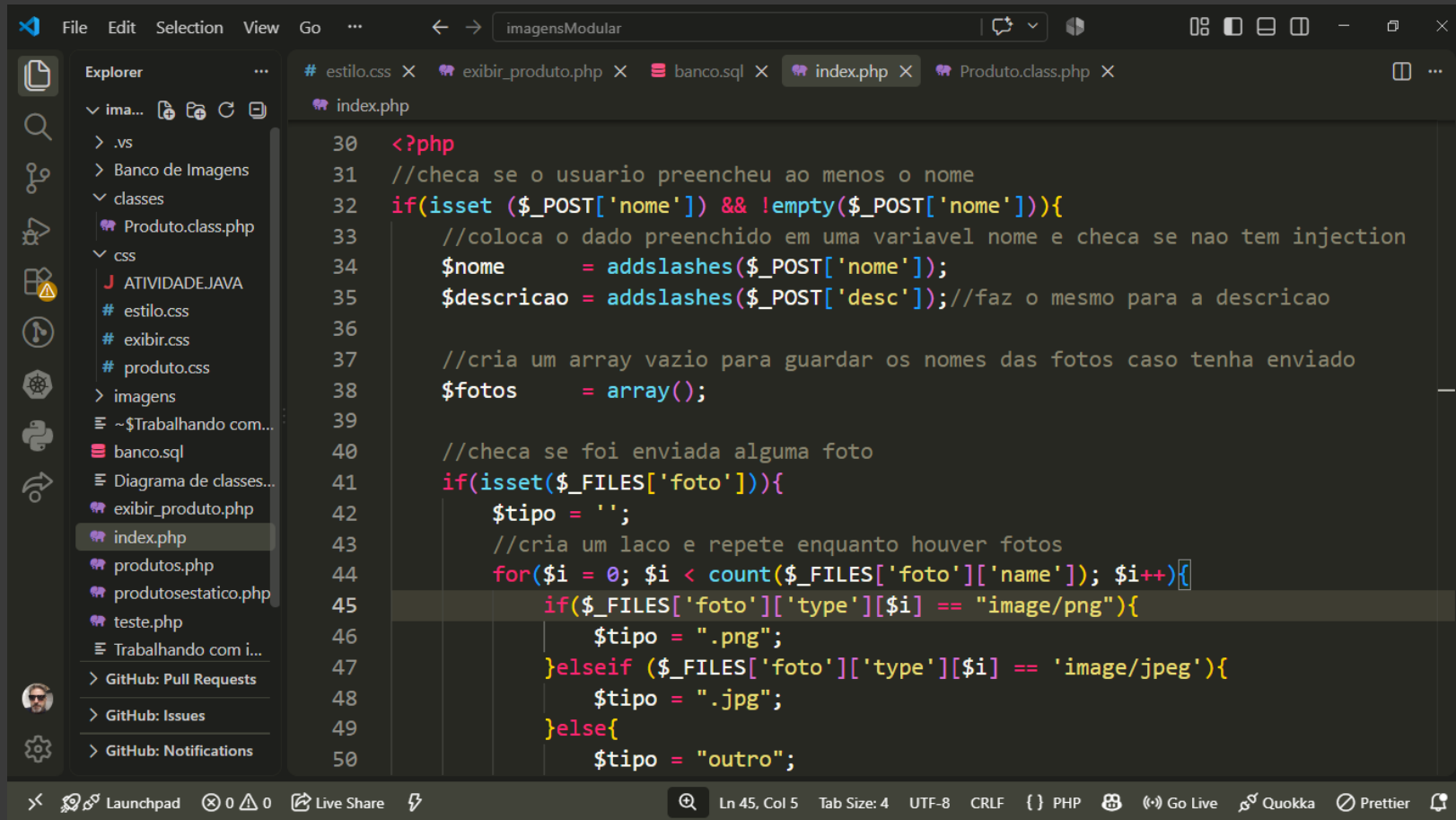
```
3  class Produto{
24
25
26      public function enviarProduto($nome, $descricao, $fotos = array()){
27          //inserir Produto na tabela produtos
28          //=====
29
30          $sql = "INSERT INTO produtos SET descricao = :d, nome_produto = :n";
31          $sql = $this->pdo->prepare($sql);
32          $sql ->bindValue(":d", $descricao);
33          $sql ->bindValue(":n", $nome);
34
35          $isOk = $sql->execute();
36
37          if($isOk == true){
38              $id_produto = $this->pdo->LastInsertId();
39          }
40
41          //inserir Imagem na tabela imagens
42          //=====
43
```

Aqui o método para enviar produtos e imagens para as tabelas. Nessa primeira parte ele insere um produto na tabela e guarda o id na variável \$id_produto para levar para a tabela imagem



```
3  class Produto{
26  public function enviarProduto($nome, $descricao, $fotos = array()){
40
41      //inserir Imagem na tabela imagens
42      //=====
43      if(count($fotos) > 0){
44          for($i = 0; $i < count($fotos); $i++){
45              $nome_foto = $fotos[$i];
46              echo"<br>";
47
48              $sql = "INSERT INTO imagens (nome_imagem, fk_id_produto) values(:n,:";
49              $sql = $this->pdo->prepare($sql);
50              $sql->bindValue(":n" , $nome_foto);
51              $sql->bindValue(":fk", $id_produto);
52
53              $isOk = $sql->execute();
54
55              return $isOk;
56          }
57      }
58  }
```

Aqui insere o nome da imagem e o id do produto para todas as imagens enviadas, se houver.



```
30 <?php
31 //checa se o usuario preencheu ao menos o nome
32 if(isset($_POST['nome']) && !empty($_POST['nome'])){
33     //coloca o dado preenchido em uma variavel nome e checa se nao tem injection
34     $nome = addslashes($_POST['nome']);
35     $descricao = addslashes($_POST['desc']); //faz o mesmo para a descricao
36
37     //cria um array vazio para guardar os nomes das fotos caso tenha enviado
38     $fotos = array();
39
40     //checa se foi enviada alguma foto
41     if(isset($_FILES['foto'])){
42         $tipo = '';
43         //cria um laço e repete enquanto houver fotos
44         for($i = 0; $i < count($_FILES['foto']['name']); $i++){
45             if($_FILES['foto']['type'][$i] == "image/png"){
46                 $tipo = ".png";
47             }elseif ($_FILES['foto']['type'][$i] == 'image/jpeg'){
48                 $tipo = ".jpg";
49             }else{
50                 $tipo = "outro";
```

Para testar esse método, voltar ao arquivo index e fazer a logica.

No form a gente não colocou um action, de forma que os dados vão permanecer nessa pagina mesmo.

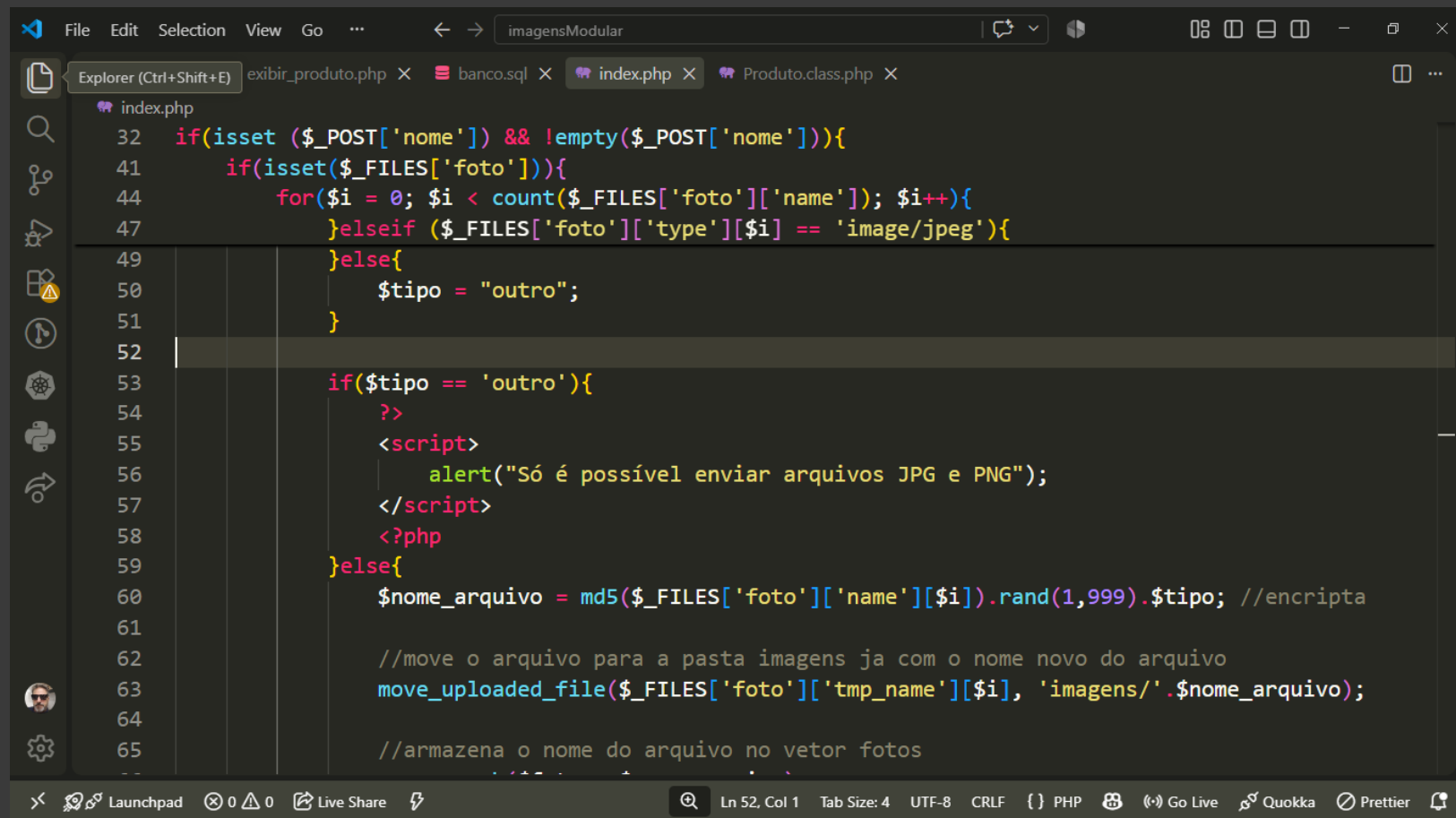
Checar se o usuário clicou no botão com o isset.

Criar um array para guardar o nome das fotos.

isset(\$_FILES) checa se o usuário selecionou alguma imagem.

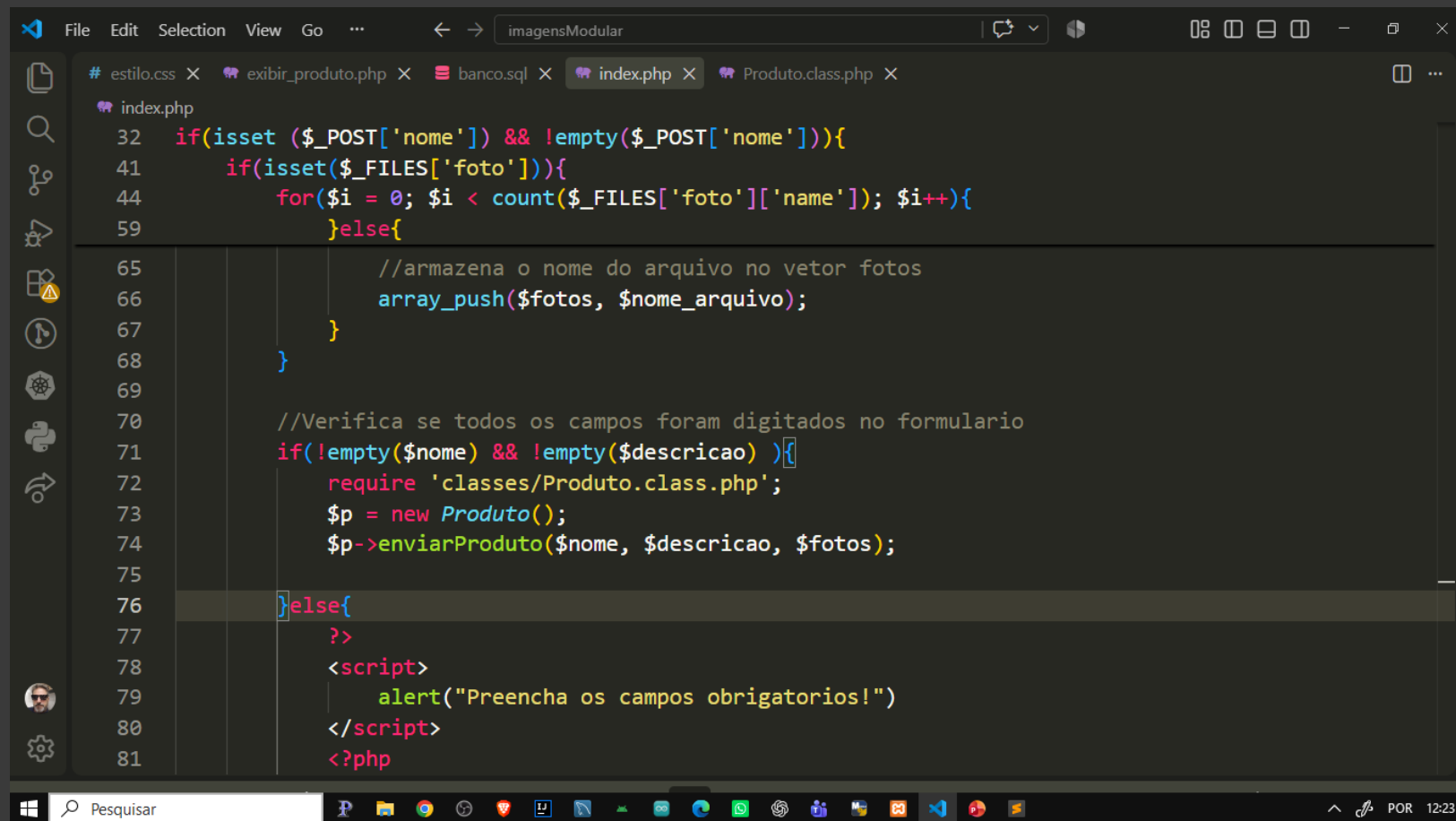
Para cada foto, checar a extensão para poder enviar apenas PNG e JPG.

Para isso criei uma variável chamada \$tipo para guardar a extensão de cada foto.



```
File Edit Selection View Go ...
imagensModular
Explorer (Ctrl+Shift+E) exibir_produto.php x banco.sql x index.php x Produto.class.php x
index.php
32 if(isset($_POST['nome']) && !empty($_POST['nome'])){
41     if(isset($_FILES['foto'])){
44         for($i = 0; $i < count($_FILES['foto']['name']); $i++){
47             }elseif($_FILES['foto']['type'][$i] == 'image/jpeg'){
49             }else{
50                 $tipo = "outro";
51             }
52
53             if($tipo == 'outro'){
54                 ?>
55                 <script>
56                 alert("Só é possível enviar arquivos JPG e PNG");
57                 </script>
58                 <?php
59             }else{
60                 $nome_arquivo = md5($_FILES['foto']['name'][$i]).rand(1,999).$tipo; //encripta
61
62                 //move o arquivo para a pasta imagens ja com o nome novo do arquivo
63                 move_uploaded_file($_FILES['foto']['tmp_name'][$i], 'imagens/'.$nome_arquivo);
64
65                 //armazena o nome do arquivo no vetor fotos
```

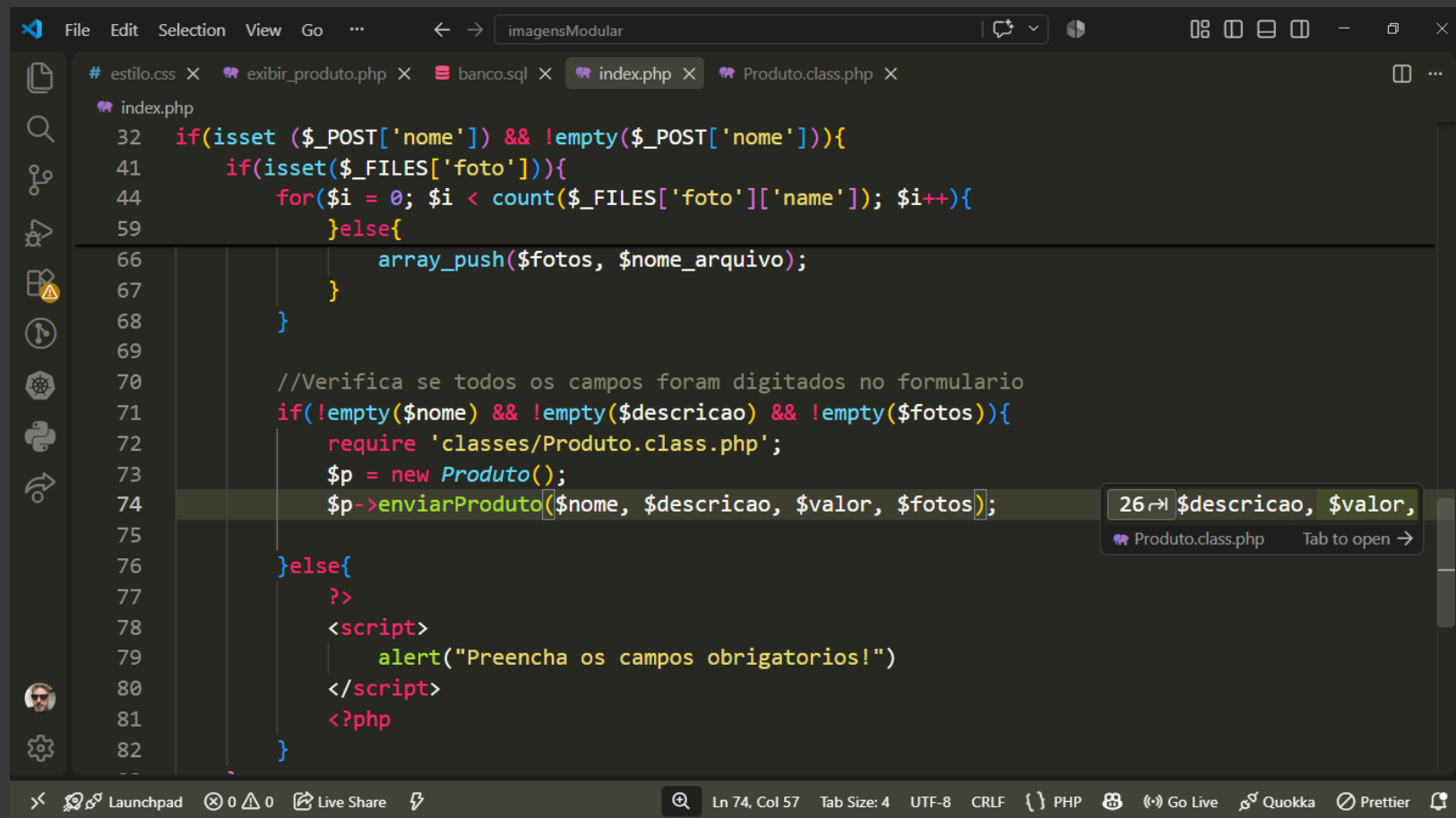
Quando salvamos um arquivo com o mesmo nome, ele sobrepõe. Para evitar isso, estou usando um numero randômico no nome. Estou também criptografando o nome. Depois junto a extensão. 7º método move_uploaded_file pega a imagem que esta na pasta temporária e salva na pasta imagens do projeto com o nome já alterado



```
File Edit Selection View Go ...
imagensModular

# estilo.css x exibir_produto.php x banco.sql x index.php x Produto.class.php x
index.php
32 if(isset($_POST['nome']) && !empty($_POST['nome'])){
41     if(isset($_FILES['foto'])){
44         for($i = 0; $i < count($_FILES['foto']['name']); $i++){
59             }else{
65                 //armazena o nome do arquivo no vetor fotos
66                 array_push($fotos, $nome_arquivo);
67             }
68         }
69
70         //Verifica se todos os campos foram digitados no formulario
71         if(!empty($nome) && !empty($descricao) ){
72             require 'classes/Produto.class.php';
73             $p = new Produto();
74             $p->enviarProduto($nome, $descricao, $fotos);
75
76         }else{
77             ?>
78             <script>
79                 alert("Preencha os campos obrigatorios!")
80             </script>
81             <?php
```

Array push adiciona no vetor o nome da imagem.



```
# estilo.css x exibir_produto.php x banco.sql x index.php x Produto.class.php x
index.php
32 if(isset($_POST['nome']) && !empty($_POST['nome'])){
41     if(isset($_FILES['foto'])){
44         for($i = 0; $i < count($_FILES['foto']['name']); $i++){
59             }else{
66                 array_push($fotos, $nome_arquivo);
67             }
68         }
69
70         //Verifica se todos os campos foram digitados no formulario
71         if(!empty($nome) && !empty($descricao) && !empty($fotos)){
72             require 'classes/Produto.class.php';
73             $p = new Produto();
74             $p->enviarProduto($nome, $descricao, $valor, $fotos);
75         }else{
76             ?>
77             <script>
78                 alert("Preencha os campos obrigatorios!")
79             </script>
80             <?php
81
82         }
```

26 ↗ \$descricao, \$valor,
Produto.class.php Tab to open →

Launchpad 0 0 Live Share Ln 74, Col 57 Tab Size: 4 UTF-8 CRLF PHP Go Live Quokka Prettier

Para gravar o registro, certificar que todos os campos foram digitados. Depois importar a classe, instanciar e acessar o método enviarProduto com os dados do formulario

Professores:

Fabio Claret
Johathas Cavalaro